

MACHINE LEARNING LAB

Linear Regression • Logistic Regression • KNN Classifier • SVM

Time	Segment	Details
Duration	3 Hours	Hands-on lab session (theory recap + coding)
Language	Python 3	Jupyter Notebook / Google Colab recommended
Libraries	numpy, pandas, matplotlib, scikit-learn	Install via pip if not already available

Lab Objectives

- Understand and implement Linear Regression for a continuous prediction problem.
- Understand and implement Logistic Regression for binary classification.
- Implement the K-Nearest Neighbors (KNN) classification algorithm and tune the value of K.
- Implement Support Vector Machine (SVM) classification with different kernels.
- Evaluate models using appropriate metrics (MSE, R^2 , accuracy, confusion matrix, classification report).

Environment Setup

Run the following command once before starting (skip if using Google Colab, these libraries are pre-installed):

```
pip install numpy pandas matplotlib scikit-learn
```

Suggested Time Allocation (Total: 3 Hours)

Time	Segment	Details
0:00 - 0:10	Setup & Introduction	Environment check, dataset overview
0:10 - 0:45	Part 1: Linear Regression	Theory recap + code + exercise
0:45 - 1:20	Part 2: Logistic Regression	Theory recap + code + exercise
1:20 - 1:30	Break	Short break
1:30 - 2:05	Part 3: KNN Classifier	Theory recap + code + exercise
2:05 - 2:45	Part 4: SVM	Theory recap + code + exercise
2:45 - 3:00	Wrap-up	Doubt clearing, submission of lab file

Part 1: Linear Regression

1.1 Concept

Linear Regression models the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a straight line (or hyperplane) that minimizes the sum of squared errors between predicted and actual values. The model equation is $y = m_1x_1 + m_2x_2 + \dots + c$.

1.2 Dataset

We will use the built-in Diabetes dataset from scikit-learn (442 patients, 10 baseline features such as age, BMI, blood pressure; target is a quantitative measure of disease progression one year after baseline). It is ideal for regression practice because it is small, clean, and already numeric.

Note: You may instead use any CSV dataset (e.g. a house-price or salary dataset) — just replace the data-loading cell; the rest of the pipeline stays identical.

1.3 Code

```
# ---- Part 1: Linear Regression ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load dataset
data = load_diabetes(as_frame=True)
df = data.frame
print(df.head())
print(df.shape)

# 2. Select a single feature for a simple illustrative model (BMI)
X = df[['bmi']].values          # feature: body mass index
y = df['target'].values         # target: disease progression

# 3. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 4. Train the model
model = LinearRegression()
model.fit(X_train, y_train)

# 5. Predict and evaluate
y_pred = model.predict(X_test)
print('Coefficient (slope):', model.coef_[0])
print('Intercept:', model.intercept_)
print('MSE:', mean_squared_error(y_test, y_pred))
print('R2 Score:', r2_score(y_test, y_pred))

# 6. Visualize
plt.scatter(X_test, y_test, color='blue', label='Actual')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Predicted line')
plt.xlabel('BMI (standardized)')
plt.ylabel('Disease progression')
plt.title('Linear Regression: BMI vs Disease Progression')
plt.legend()
plt.show()
```

```
# 7. Bonus: Multiple Linear Regression using ALL features
X_all = df.drop('target', axis=1).values
X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X_all, y, test_size=0.2, random_state=42)
model_multi = LinearRegression().fit(X_train2, y_train2)
pred_multi = model_multi.predict(X_test2)
print('Multiple LR R2 Score:', r2_score(y_test2, pred_multi))
```

Exercise 1

1. Repeat the single-feature model using a different feature (e.g. 's5' or 'bp') instead of 'bmi'. Which feature gives a better R^2 score?
2. Compare the R^2 of the single-feature model with the multiple-feature model. Explain the difference in 2-3 lines.
3. Load your own CSV dataset (e.g. Salary vs Experience) with `pandas.read_csv()` and repeat steps 1-6.

Part 2: Logistic Regression

2.1 Concept

Logistic Regression is a classification algorithm (despite the name) used to predict a categorical outcome, typically binary (0/1). It passes a linear combination of inputs through the sigmoid function to output a probability between 0 and 1, which is then thresholded (commonly at 0.5) to obtain a class label.

2.2 Dataset

We will use the Breast Cancer Wisconsin dataset (built into scikit-learn): 569 samples, 30 numeric features computed from digitized images of breast masses, target is malignant (0) or benign (1). This is a standard, real, well-labeled binary classification dataset.

2.3 Code

```
# ---- Part 2: Logistic Regression ----
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load dataset
data = load_breast_cancer(as_frame=True)
df = data.frame
print(df.shape)
print(data.target_names)  # ['malignant' 'benign']

# 2. Features and target
X = df.drop('target', axis=1)
y = df['target']

# 3. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 4. Feature scaling (important for logistic regression convergence)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Train the model
clf = LogisticRegression(max_iter=5000)
clf.fit(X_train_scaled, y_train)

# 6. Predict and evaluate
y_pred = clf.predict(X_test_scaled)
print('Accuracy:', accuracy_score(y_test, y_pred))
print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))
print('Classification Report:\n', classification_report(y_test, y_pred))

# 7. Predict probability for a single new sample
sample = X_test.iloc[[0]]
sample_scaled = scaler.transform(sample)
prob = clf.predict_proba(sample_scaled)
print('Predicted probabilities [malignant, benign]:', prob)
```

Exercise 2

1. Change `test_size` to 0.3 and re-run. Does accuracy change significantly?
2. Remove the `StandardScaler` step and re-run. What happens to the model performance / convergence warnings?
3. Print the confusion matrix as a heatmap using seaborn (`sns.heatmap`).

Part 3: K-Nearest Neighbors (KNN) Classifier

3.1 Concept

KNN is a simple, non-parametric, instance-based classification algorithm. To classify a new point, it looks at the 'K' closest points in the training data (using a distance metric, usually Euclidean) and assigns the majority class among those neighbors. The choice of K controls the bias-variance trade-off: small K → low bias/high variance (overfitting), large K → high bias/low variance (underfitting).

3.2 Dataset

We will use the classic Iris dataset (built into scikit-learn): 150 samples of iris flowers, 4 features (sepal length/width, petal length/width), 3 classes (setosa, versicolor, virginica). It is small, well-separated, and ideal for visualizing distance-based classifiers.

3.3 Code

```
# ---- Part 3: KNN Classifier ----
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load dataset
data = load_iris(as_frame=True)
df = data.frame
print(df.head())
print(data.target_names)  # ['setosa' 'versicolor' 'virginica']

# 2. Features and target
X = df.drop('target', axis=1)
y = df['target']

# 3. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 4. Feature scaling (KNN is distance-based, scaling matters a lot)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Train KNN with K = 5
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)

# 6. Predict and evaluate
y_pred = knn.predict(X_test_scaled)
print('Accuracy (K=5):', accuracy_score(y_test, y_pred))
print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))
print('Classification Report:\n', classification_report(y_test, y_pred))

# 7. Find the best K by testing K = 1..20
accuracies = []
k_range = range(1, 21)
for k in k_range:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_scaled, y_train)
    acc = accuracy_score(y_test, model.predict(X_test_scaled))
```

```
    accuracies.append(acc)

plt.plot(k_range, accuracies, marker='o')
plt.xlabel('Value of K')
plt.ylabel('Test Accuracy')
plt.title('KNN: Accuracy vs K')
plt.xticks(k_range)
plt.grid(True)
plt.show()

best_k = k_range[accuracies.index(max(accuracies))]
print('Best K:', best_k, 'with accuracy:', max(accuracies))
```

Exercise 3

1. Re-run the model using only 2 features (petal length & petal width) and plot the decision boundary.
2. Try distance metric 'manhattan' instead of the default 'euclidean' (weights='distance', metric='manhattan') — does accuracy change?
3. Explain, in your own words, why scaling matters for KNN but matters less for a decision tree.

Part 4: Support Vector Machine (SVM)

4.1 Concept

SVM finds the hyperplane that best separates classes by maximizing the margin between the closest points of each class (the support vectors). For data that is not linearly separable, SVM uses the 'kernel trick' (e.g. RBF, polynomial) to project data into a higher-dimensional space where a linear separator exists.

4.2 Dataset

We continue with the Iris dataset (for consistency and easy 2D visualization of decision boundaries), and additionally show SVM on the Breast Cancer dataset to compare with Part 2's Logistic Regression results.

4.3 Code

```
# ---- Part 4: SVM Classifier ----
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# 1. Load dataset (use only 2 features for visualization: petal length & width)
data = load_iris()
X = data.data[:, 2:4]      # petal length, petal width
y = data.target

# 2. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Train SVM with RBF kernel
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale')
svm_rbf.fit(X_train_scaled, y_train)
y_pred = svm_rbf.predict(X_test_scaled)
print('RBF kernel accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=data.target_names))

# 5. Compare kernels
for k in ['linear', 'poly', 'rbf', 'sigmoid']:
    m = SVC(kernel=k)
    m.fit(X_train_scaled, y_train)
    acc = accuracy_score(y_test, m.predict(X_test_scaled))
    print(f'Kernel={k:8s} -> Accuracy={acc:.3f}')

# 6. Visualize decision boundary (RBF kernel)
def plot_decision_boundary(model, X, y):
    h = 0.02
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                        np.arange(y_min, y_max, h))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    plt.contourf(xx, yy, Z, alpha=0.3, cmap='viridis')
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolor='k', cmap='viridis')
```

```
plt.xlabel('Petal length (scaled)')
plt.ylabel('Petal width (scaled)')
plt.title('SVM (RBF kernel) Decision Boundary')
plt.show()
```

```
plot_decision_boundary(svm_rbf, X_train_scaled, y_train)
```

Exercise 4

1. Change C from 1.0 to 0.01 and then to 100. Observe and note how the decision boundary and accuracy change.
2. Repeat the SVM (all features, kernel='rbf') on the Breast Cancer dataset from Part 2 and compare its accuracy with Logistic Regression's accuracy.
3. In your own words, explain what the parameter 'gamma' controls in the RBF kernel.

Lab Submission Guidelines

- Submit a single Jupyter Notebook (.ipynb) containing all 4 parts with outputs (plots + printed metrics) visible.
- Answer all exercise questions in markdown cells directly below the relevant code.
- Name the file as: RollNumber_Name_MLLab_Session.ipynb
- Submit before the end of the lab session through the designated portal / email.

Quick Reference: Algorithm Summary

Time	Segment	Details
Linear Regression	Diabetes dataset	Predicts continuous value; evaluated with MSE / R^2
Logistic Regression	Breast Cancer dataset	Binary classification; sigmoid + threshold
KNN	Iris dataset	Distance-based; majority vote of K neighbors
SVM	Iris / Breast Cancer	Maximum-margin hyperplane; kernel trick for non-linear data